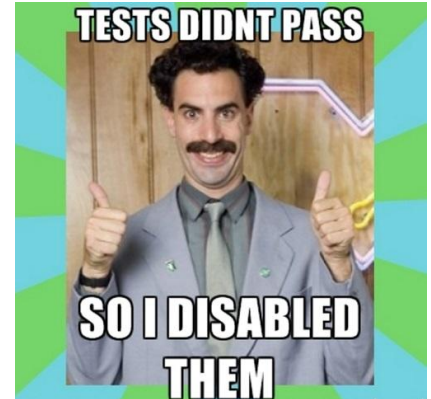


Unit Testing and Mocking in Java

COMS-4156 ASE



What is Unit Testing and Why Do We Do It?

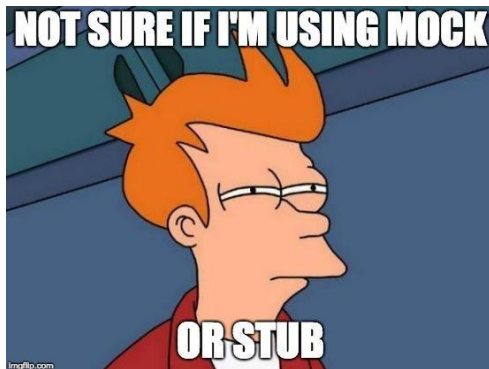
- Testing the smallest parts of a software (units) to ensure they work as intended
- Want to “break” the code to squash bugs before deploying to prod
- Branch Coverage: measures how many branches of program control have been executed during testing
 - 2nd iteration -> 85% coverage (so try to get started in 1st iteration)
- Ideally, every part of your code has been tested. This isn't practical, but maximizing test coverage minimizes the chances of things going awry down the line (especially want to avoid this in prod).
- Test “cycle”: build, test, fix bugs, repeat

What is Mocking and Why Do We Do It?

- Mocking is the process of creating fake versions of external or internal services to simulate their behavior.
- Allows devs to isolate code being tested from dependencies (i.e. you assume everything else works as intended) - in order to focus on the new functionality you're trying to develop
- Helps tests run more quickly and reliably (uses same dummy data, etc.)
- Example: Your Car class depends on an Engine class for several methods. To test Car, you would mock the Engine class to ensure you can focus on testing the Car class methods.

Mocking vs Stubbing

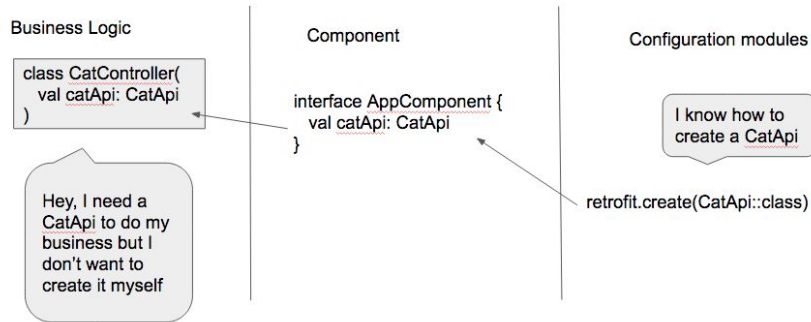
- Mock:
 - Set up data
 - Set up expectations
 - Test
 - Verify expectations
 - Verify state
 - Teardown
- HOW has the result been achieved?
- Stub:
 - Set up
 - Test
 - Verify state
 - Teardown
- Has predetermined behavior



Quick Aside: Dependency Injection

- Design pattern that allows you to decouple objects and their dependencies
- Makes code generally cleaner, more maintainable, and more testable (you should consider it but it's not a requirement)
 - Generally, loose coupling is considered good practice so that you don't get a “daisy chain of disaster” when things break

Dependency Injection Architecture



Unit Testing in Java

Relevant Tools for Unit Testing in Java

- JUnit: allows you to write unit tests for Java code (IntelliJ has built-in integration)
- Mockito: allows you to mock interfaces
- Spring Test MockMvc: allows you to test the web (API) layer of a service
- JaCoCo: provides code coverage reports

Steps for Unit Testing

1. Precondition (setup that needs to be done for the test)
2. Action (do the thing)
3. Verification (ensure that the result was as expected)

You still need to do other tests, like integration and API testing, otherwise your SW will look (and act) like this ->



Why Do We Do Setup + Teardown?

Demo

(forked from @lakshitha200)

Key Takeaways

- Unit testing is an important part of any software development cycle
 - BUT, it should not be the only type of testing performed
- Unit testing allows us to test the smallest parts of our code to make sure that they function independently
- You should aim to test as many units as possible with as much branch coverage as possible
 - Don't only test happy paths because you don't know what will occur in prod

Additional Resources

- [Mockito tutorial](#) (Tutorialspoint is great for Java stuff in general)
- [Spring.io guide to testing web layer \(Mockmvc\)](#)
- [How to set up unit testing in Java](#)

When in doubt, schedule some time with your team's mentor to ensure you are on the right track.

Questions?